

Intelligent Bug Diagnosis Platform with Fix Recommendation Assistance: A Multi-Agent, Retrieval-Augmented Approach to Automated Software Defect Triage and Remediation

Manyam Geetha Sree GenAI Intern, SURE Trust

Abstract

Manual bug triage and diagnosis remain a significant bottleneck in software maintenance: engineers must read raw bug reports and stack traces, judge severity and priority, search for prior occurrences of the same defect, and propose a fix — largely from memory or ad-hoc search. This report presents the **Intelligent Bug Diagnosis Platform**, an AI-assisted system that automates this workflow using a five-agent pipeline grounded in a Retrieval-Augmented Generation (RAG) knowledge base of historical software defects. The system accepts a bug report, stack trace, or error log as input; classifies its severity, priority, and affected component; extracts structured diagnostic information from the raw error text; retrieves semantically similar historical defects from a knowledge base of over 56,000 records sourced from the Mozilla, Apache, and Eclipse bug repositories; checks for duplicate live submissions; and produces a structured, evidence-grounded fix recommendation. Critically, the knowledge base grows over time: once a recommended fix is confirmed to work, it is appended back into the retrieval corpus, allowing future diagnoses to draw on the system's own accumulated experience. Component-level validation shows 60% exact-match / 83.3% adjacent-accuracy for severity classification and 81.8% fair-match accuracy for structured error-type extraction across eleven programming languages. End-to-end pipeline testing on a stratified sample of 39 real bug reports showed zero hallucinated fixes and a 68.75% exact-fix-match rate among completed test cases, with the remainder blocked only by external API rate limits rather than pipeline defects. This report describes the system's architecture, agent design, knowledge base construction and growth mechanism, and empirical evaluation, and discusses its limitations and directions for future work.

Index Terms — Retrieval-Augmented Generation, Multi-Agent Systems, Large Language Models, Software Defect Triage, Bug Report Analysis, Semantic Similarity Search, Automated Fix Recommendation.

I. Introduction

A. Motivation

Software defect management is a recurring, high-volume task in any engineering organization. Every incoming bug report — whether filed by an end user, generated by an automated test suite, or surfaced by a runtime crash log — must be read, classified by severity and urgency, routed to the correct component owner, checked against previously reported issues to avoid duplicated investigation effort, and ultimately diagnosed and fixed. In practice, this process is slow, inconsistent across engineers, and wasteful of institutional knowledge: resolutions to past defects are typically archived in closed tickets that are not searchable in any structured or reusable way, so semantically identical classes of bugs are frequently re-diagnosed from scratch.

B. Problem Statement

The objective of this project is to design and implement a system that:

1. Accepts an unstructured bug report, stack trace, or error log as input.
2. Automatically classifies the report by severity, priority, and affected software component.
3. Extracts structured diagnostic facts (error type, failure location, call/execution path) from raw, often messy error text spanning multiple programming languages.
4. Retrieves semantically similar defects from a large historical knowledge base to ground its reasoning in real precedent, rather than relying purely on a language model's unverified prior knowledge.
5. Detects whether the incoming report duplicates an issue already submitted to the live system.
6. Synthesizes all of the above into a concrete, actionable fix recommendation with supporting evidence.
7. Improves over time by incorporating confirmed-successful fixes back into its own knowledge base.

C. Contributions

This work makes the following contributions:

- A **five-agent pipeline architecture** in which each agent has a single, well-scoped responsibility, communicating through a shared structured schema rather than a single monolithic prompt.
 - A **RAG-grounded root-cause and remediation strategy** that retrieves the top-N most similar historical defects (via sentence-embedding cosine similarity) before an LLM is asked to reason about a novel bug, reducing unsupported speculation.
 - A **self-improving knowledge base**: confirmed fixes are embedded and appended to the retrieval corpus using the same schema and embedding model as the original historical data, without recomputing existing embeddings.
 - A **duplicate-detection design that is deliberately decoupled from the historical knowledge base**, comparing only against prior *live* submissions, so that a newly submitted bug cannot spuriously match itself or match unconfirmed historical noise.
 - An **empirical, per-component evaluation methodology**, including a fuzzy-match scoring scheme developed specifically because exact-string matching unfairly penalizes semantically correct but differently-worded LLM outputs.
-

II. Background and Related Concepts

A. Bug Report Structure

A typical bug report combines free-text description, a severity/priority label, an affected component, and — for runtime failures — a stack trace or compiler error message. Historical bug-tracking datasets (e.g., Bugzilla exports from Mozilla, Apache, and Eclipse) provide large volumes of resolved reports with the true resolution and severity recorded, making them suitable as a grounding corpus.

B. Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) couples a large language model with a retrieval step over an external corpus, so that generation is conditioned on retrieved evidence rather than the model's parametric memory alone. In this system, RAG is applied twice: once to retrieve similar historical defects

for root-cause hypothesis generation, and once (over a separate, smaller corpus of live submissions) for duplicate detection.

C. Semantic Similarity via Sentence Embeddings

Rather than relying on keyword overlap, the system encodes both bug descriptions and query text using the `all-MiniLM-L6-v2` sentence-embedding model and measures relatedness via cosine similarity. This allows retrieval of defects that are conceptually related even when their surface wording differs substantially.

III. System Architecture

A. End-to-End Data Flow

The system is organized as a linear pipeline with a feedback loop, structured as follows:

1. **User Submission** — a bug report is submitted as pasted text or file upload through the front-end interface.
2. **Triage + Log Analysis** — the report is classified for severity/priority/component and its raw error text is parsed into structured diagnostic fields.
3. **Retrieval (RAG)** — the parsed report is embedded and used to query the historical defect knowledge base for the top-N most similar prior defects.
4. **Root Cause Agent** — reasons over the retrieved evidence to produce a grounded root-cause hypothesis.
5. **Duplicate Detection** — the report is separately compared against previously submitted *live* reports (not the historical KB) to flag likely duplicates.
6. **Remediation Agent** — synthesizes triage, log analysis, root-cause, and duplicate-detection outputs into a structured fix recommendation.
7. **Structured Findings Display** — results are rendered to the user.
8. **User Confirmation** — the user reports whether the recommended fix worked.
9. **Knowledge Base Growth** — confirmed-successful fixes are embedded and appended to the historical knowledge base, closing the feedback loop.

A key persistence design decision is that a newly submitted bug is written to the live-submissions store **only after** duplicate detection has run against it, ensuring a bug can never be compared against itself.

B. Technology Stack

Layer	Technology	Role
Language	Python	End-to-end implementation
User Interface	Streamlit	Bug submission and analytics dashboard
LLM Inference	Groq API, <code>llama-3.3-70b-versatile</code> (agent reasoning); <code>llama-3.1-8b-instant</code> used during early prototyping	Powers all agent reasoning calls
Embeddings	<code>sentence-transformers</code> (<code>all-MiniLM-L6-v2</code>)	Encodes bug text for semantic retrieval
Similarity Search	scikit-learn (<code>cosine_similarity</code>), in-memory over saved embeddings	Historical KB retrieval and duplicate detection
Chunking	LangChain text splitters	Historical KB text preprocessing
Clustering	<code>AgglomerativeClustering</code>	Root-cause pattern analytics
Persistence	SQLite (live submissions) + <code>.npy</code> / <code>.csv</code> (embeddings and metadata)	Durable storage for submissions and the growing KB
Data Processing	pandas, NumPy	Dataset cleaning, aggregation, and analysis
Visualization	Plotly	Analytics dashboard

IV. Multi-Agent Pipeline Design

Each agent is implemented as an independent function with a tightly scoped prompt, a fixed JSON output schema, and retry-with-backoff error handling for LLM rate limits. This section describes each agent's responsibility and prompt-engineering approach.

A. Triage Agent

Responsibility: classify severity (Critical/High/Medium/Low), priority (Immediate/High/Medium/Low), and the affected software component, with a confidence score and short justification.

Design note: an early version of the system prompt produced severity classifications that were miscalibrated toward the extremes. The prompt was revised to include an explicit calibration instruction — stating that the majority of real-world bugs are routine Medium severity — together with worked examples spanning all four severity levels, so that the model does not over-classify ordinary bugs as urgent merely because the report mentions words such as "error" or "exception."

Ground-truth mapping. The historical dataset's native severity vocabulary (`blocker`, `critical`, `major`, `normal`, `minor`, `trivial`, `unknown`, in mixed case) was normalized and mapped onto the four-level output scale (`blocker`/`critical` → `Critical`, `major` → `High`, `normal`/`unknown` → `Medium`, `minor`/`trivial` → `Low`) to allow like-for-like evaluation against the LLM's predictions.

B. Log Analysis Agent

Responsibility: parse raw, unstructured stack traces, tracebacks, or compiler error messages — optionally alongside the offending source code — into four structured fields: a **specific** error type (e.g., "Dereferencing Void Pointer" rather than the generic "Compilation Error"), the exact failure location, the function/method call chain leading to the failure, and a confidence score reflecting how unambiguous the input was.

Design note: the prompt was iterated over several revisions to (a) push the model toward specific, compiler/runtime-accurate error names rather than broad categories, and (b) enforce strict JSON-escaping of double-quote characters appearing inside code snippets (a frequent source of JSON parse failures, particularly for Rust `println!` macros and Java string literals). Confidence scoring was explicitly instructed to vary realistically with input clarity rather than defaulting to a near-1.0 value for every input.

C. Root Cause Agent

Responsibility: given the triage and log-analysis output for a new bug plus the top-N historical defects retrieved via embedding similarity, produce a root-cause hypothesis, a confidence score, and a list of supporting historical bug identifiers with short justification summaries.

Design note: the prompt is explicitly instructed to acknowledge weak or absent historical grounding rather than fabricate confident connections — worked examples include cases with no closely related historical bug, in which the model is expected to lower its confidence score and state that no strong precedent was found, rather than forcing a tenuous match.

D. Duplicate Detection Agent

Responsibility: determine whether a newly submitted bug is a likely duplicate of a bug already present in the live-submissions store.

Design note: duplicate scoring is computed purely mathematically via cosine similarity over sentence embeddings — **not** by LLM judgment — using fixed thresholds (similarity ≥ 0.90 labeled `same`; 0.75–0.89 labeled `similar`; below 0.75 discarded as unrelated). The LLM's only role is to produce a short, human-readable explanation of *why* two matched reports appear to describe the same underlying issue; it is explicitly instructed not to alter or second-guess the numerically assigned label. This agent queries the live-submissions database only, deliberately excluded from the historical knowledge base, so that duplicate detection reflects genuine repeat reporting rather than similarity to unconfirmed archival data.

E. Remediation Agent

Responsibility: synthesize the bug's severity/component/error metadata, the root-cause hypothesis and its supporting historical evidence, and any duplicate-detection result into a final structured recommendation: a recommended fix summary, ordered fix steps, a before/after code example, validation steps to confirm the fix, a prevention tip, a confidence score, a reasoning statement, and a list of the specific references used to ground the recommendation.

Design note: when a near-exact duplicate is found, the agent is instructed to reference the existing fix rather than independently re-deriving a new one, avoiding redundant engineering effort. When no strong historical or duplicate evidence exists, the agent is instructed to lower its confidence score accordingly rather than presenting a generic best-practice suggestion with unwarranted certainty.

V. Historical Defect Knowledge Base

A. Data Sources and Record Schema

The knowledge base aggregates publicly available bug-tracking export datasets from the **Mozilla**, **Apache**, and **Eclipse** projects (obtained via Kaggle), combined with a synthetic, language-labeled stack-trace/error dataset spanning eleven programming languages (Python, Java, JavaScript, TypeScript, C, C++, Go, Rust, PHP, HTML, CSS) used primarily to validate the Log Analysis Agent. Each knowledge base record is stored with the fields: `bug_id`, `title`, `description`, `severity`, `resolution`, `status`, and `source_dataset`. For retrieval, records are embedded as the concatenation of title and description (capped and chunked for length) using the `all-MiniLM-L6-v2` model; the resolution text itself is retained as metadata only and is never embedded, so that retrieval is driven by problem similarity rather than solution-text similarity.

B. Embedding and Retrieval Strategy

At query time, an incoming bug's combined description is encoded with the same embedding model, and cosine similarity is computed in-memory against the full saved embedding matrix (scikit-learn) to retrieve the top-N most similar historical records for the Root Cause Agent.

C. Knowledge Base Growth Mechanism

A distinguishing feature of the system is that it does not treat its knowledge base as static. When a user confirms that a recommended fix worked, the confirmed problem/fix pair is formatted using the same schema as the original historical data, embedded with the same model, and appended to the stored embedding matrix and metadata table — tagged with `source_dataset = live_submissions` — without recomputing any of the pre-existing embeddings. Confirmation is tracked using a three-state resolution status:

- `resolved_added_to_kb` — the fix was confirmed working, and the append to the knowledge base succeeded.
- `resolved_kb_append_failed` — the fix was confirmed working, but the underlying write operation failed; the human confirmation itself is still preserved.
- `unresolved` — the fix did not work, and nothing is added to the knowledge base.

This separation reflects a deliberate design principle: whether a fix worked and whether a storage write succeeded are independent facts, and conflating them would silently corrupt the record of what the user actually reported.

VI. Implementation Details

The pipeline is orchestrated in Python, with each agent implemented as a discrete function wrapped in retry-with-exponential-backoff logic to tolerate LLM API rate limiting. All agents communicate through explicit JSON schemas rather than free text, which allows their outputs to be composed directly into the input of the next stage (e.g., the Root Cause Agent's retrieved evidence and the Duplicate Detection

Agent's match result are both passed as structured input to the Remediation Agent). Live submissions and their triage/log-analysis/root-cause outcomes are persisted in a SQLite database; the historical knowledge base's embeddings and metadata are persisted as a NumPy array and CSV file respectively, loaded once at startup and updated incrementally by the growth mechanism.

VII. Experimental Evaluation

A. Triage Agent Validation

The Triage Agent was validated against a stratified random sample of 30 bugs drawn from the cleaned historical knowledge base, using the normalized four-level severity ground truth described in Section IV-A.

- **Exact-match accuracy:** $18/30 = 60.0\%$
- **Adjacent-level accuracy** (predicted severity within one level of ground truth): $25/30 = 83.3\%$

Inspection of the largest misses showed that most disagreements were plausible boundary cases (e.g., a "missing metadata errors on the console" report labeled Critical in the historical data but classified Medium by the agent) rather than clear misclassifications, consistent with the inherent subjectivity of severity labeling in real bug trackers.

B. Log Analysis Agent Validation

Because the agent is explicitly prompted to produce *specific* error-type names rather than generic categories (Section IV-B), naive exact-string matching against ground truth severely underestimates correctness — for example, a predicted "std::bad_alloc" versus a ground-truth "std::bad_alloc Exception" is functionally identical but would fail an exact match. A fuzzy-matching scheme was therefore developed iteratively:

1. An initial substring-containment check yielded 54.5% accuracy on a 33-bug sample.
2. A token-overlap check with noise-word removal ("error", "warning", "exception", etc.) yielded 52.7% on a larger 55-bug sample (v2).
3. A refined proportional-overlap scheme, requiring at least half of the shorter answer's meaningful words to match, yielded **81.8% (45/55)** fair-match accuracy (v3) — the methodology ultimately adopted for reporting.

Per-language breakdown under the v3 scheme showed strong performance for Python, Java, HTML, and PHP (100%), moderate performance for CSS, C, JavaScript, Go, and Rust (80%), and weaker performance for C++ (40%) and TypeScript (60%), reflecting the greater diversity of compiler-error phrasing in those ecosystems.

C. End-to-End Pipeline Testing

The full pipeline (Triage → Log Analysis → Retrieval → Root Cause → Duplicate Detection → Remediation) was evaluated on a reproducibly sampled (fixed random seed) set of 39 bug reports spanning 11 programming languages and the three historical source repositories, submitted through the live system exactly as an end user would. Each completed test case was judged on three criteria: whether the recommended fix was a hallucination unsupported by the input, the strength of historical grounding

(similarity ≥ 0.7 = strong, $0.4-0.69$ = weak, < 0.4 = none), and whether the recommended fix addressed the same underlying problem as the dataset's known correct fix.

- 16 of 39 test cases were fully completed before the LLM provider's daily API quota was exhausted; the remainder were not attempted due to this external constraint rather than any pipeline failure.
- **Zero hallucinated fixes** were observed across all completed test cases.
- **11 of 16 (68.75%)** completed cases produced a fix that exactly matched the dataset's expected fix.
- The remaining **5 of 16 (31.25%)** were partial matches (addressing the correct general problem area without exactly reproducing the expected fix); none were assessed as incorrect.
- Historical-grounding strength across the 15 cases for which a grounding judgment was recorded was: 2 strong, 11 weak, 2 none — indicating that the $\sim 56,000$ -record knowledge base provides broad but not uniformly deep coverage, particularly for languages underrepresented in its original Mozilla/Apache/Eclipse source data (notably Rust, Go, and TypeScript).

D. Knowledge Base Growth Verification

The growth mechanism (Section V-C) was verified empirically: after two test bugs were submitted, diagnosed, and confirmed as successfully resolved, the knowledge base's stored record count increased from **56,862 to 56,864**, confirming that only confirmed-resolved cases are appended. As an independent cross-check, a bug that the Duplicate Detection Agent flagged as a 100% match against a prior live submission was, separately, grouped into the same cluster by the unrelated analytics clustering pipeline — two independently computed signals converging on the same conclusion.

VIII. Discussion — Limitations and Threats to Validity

1. **API rate limiting.** The Groq on-demand tier used for development enforces a fixed daily token quota, which directly limited the size of the end-to-end test set (16 of 39 planned cases). This is an infrastructure constraint rather than a defect in the pipeline's design or logic.
 2. **Uneven historical language coverage.** Because the historical knowledge base's source repositories (Mozilla, Apache, Eclipse) predate the widespread adoption of Rust, Go, and TypeScript, root-cause grounding is measurably weaker for bugs in those languages. This gap is expected to narrow naturally as the knowledge-base growth mechanism accumulates confirmed live resolutions in underrepresented languages.
 3. **Compound submissions.** Bug reports describing more than one distinct issue in a single submission may cause the Log Analysis Agent to produce a single, comma-separated `error_type` rather than cleanly separating the constituent issues.
 4. **Knowledge base append strategy.** The current growth mechanism loads the full embedding array, appends the new vector, and re-saves the array to disk. This is computationally inexpensive at the present scale (tens of thousands of records, sub-second operation) but is not the storage strategy that would be chosen for a knowledge base several orders of magnitude larger.
 5. **Evaluation is manual.** Fix-correctness and hallucination judgments in the end-to-end evaluation (Section VII-C) were made by manual review against known-correct historical fixes rather than an automated evaluation metric, which bounds the test set size achievable within the project timeline.
-

IX. Future Work

- Development of an automated evaluation harness for remediation quality, to allow testing to scale beyond manual review once API quota constraints are lifted.
 - Migration of the knowledge base to a chunked or incremental storage format to remove the full-array-reload scaling limitation.
 - Explicit detection and handling of compound, multi-issue bug submissions, including automatic decomposition into separate diagnostic threads.
 - Controlled evaluation of embedding fix-resolution text alongside problem text (currently resolution text is excluded from embeddings by design), to determine whether this improves retrieval quality without introducing solution-leakage bias.
-

X. Conclusion

This report has described the design, implementation, and empirical evaluation of a five-agent, retrieval-augmented bug diagnosis platform. By decomposing the diagnostic workflow into narrowly scoped agents — triage, log analysis, root-cause reasoning grounded in historical retrieval, duplicate detection scoped to live submissions, and remediation synthesis — the system produces structured, evidence-backed fix recommendations rather than unconstrained LLM output. Component-level validation (60-83% severity accuracy, 81.8% structured error-type extraction accuracy) and end-to-end testing (zero hallucinations, 68.75% exact-fix accuracy on completed cases) indicate that the grounding strategy is effective, while honestly disclosed limitations — API rate constraints, uneven language coverage, and compound-submission handling — identify concrete, addressable directions for continued development. The system's core novelty, a knowledge base that grows from its own confirmed successes, positions it to improve its grounding quality over time rather than remaining a static, one-time-trained tool.

References

1. Mozilla Bugzilla public bug-tracking dataset (via Kaggle).
2. Apache Software Foundation public bug-tracking dataset (via Kaggle).
3. Eclipse Foundation public bug-tracking dataset (via Kaggle).
4. N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *Proc. EMNLP-IJCNLP*, 2019 (`all-MiniLM-L6-v2` model).
5. Groq Inc., Groq API, (`llama-3.3-70b-versatile`) model.
6. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.
7. LangChain documentation, text-splitting utilities.
8. Streamlit Inc., Streamlit framework documentation.